

左式堆与斜堆

4.1 左式堆

4.1.1 可合并堆的引入

在数据结构基础课程中，我们已经学过最简单的二叉堆（优先队列）结构，该结构的目标在于能够快速访问全局最优的值（优先值），从而有利于一些需要不断选取最优值的场景，例如操作系统中的任务调度，或者 Dijkstra 算法以及最小生成树的 Prim 算法加速等。作为一个数据结构，堆自然有如下两种性质需要规定：

1. 结构性质：二叉堆是一棵完全二叉树。并且由于是完全二叉树，故树高 h 和总结点数之间的关系显然为 $h = O(\log n)$ 。
2. 序性质：对于最大堆，根结点就是最大元素，然后每个结点的孩子必须小于等于自身；反之对于最小堆，根结点是最小元素，孩子应当大于等于父亲。

之前的平衡搜索树也是由这两者规定的，序性质大部分都是左小右大，B+ 树略复杂，结构性质则是各显神通，读者可以自行回顾总结。

此前已经学习了基于数组的二叉堆实现，事实上是利用数组构造了一棵完全二叉树，通过数组索引除以 2 找到结点的父亲，乘以 2 和乘以 2 加 1 获得结点的左右孩子，因此非常方便，数组布局紧凑，也省去了每个结点的孩子指针。下面简要回顾一下二叉堆的一些基本操作（以最小堆为例，本节无特殊说明都默认最小堆）：

1. Insert：直接插在完全二叉树的下一个空位上，然后 percolate up 找到它应当在的位置，显然最坏情况也与完全二叉树的高度成正比，即 $O(\log n)$ 。
2. FindMin：直接返回根结点即可，时间 $O(1)$ 。
3. DeleteMin：直接用完全二叉树的最后一个元素顶替根结点，然后 percolate down 找到新根结点的归宿，时间 $O(\log n)$ 。
4. BuildHeap：即对 n 个元素建堆存储。这是一个比较特别的操作，最原始的方法就是连续插入 n 次，但这样时间复杂度为 $O(n \log n)$ ，所以要有更好的手段。更好的方法是：无需管序性质，直接任意插入这 n 个值，然后从完全二叉树倒数第二排有孩子的结点开始，往前依次检查是否有违反序性质的，有就 percolate down 到正确的位置，循环直到根结点也调整完毕为止，其总成本为 $O(n)$ ：高度至少为 j 的结点至多有 $n/2^j$ 个，也就是需要下滤到第 j 步的结点至多有 $n/2^j$ 个，故总工作量至多为 $O(n \sum_{j \geq 1} 2^{-j}) = O(n)$ 。

除此之外还有一些操作，这些操作在 Dijkstra 算法加速等场景中可能有应用，因此也展开介绍：

1. DecreaseKey / IncreaseKey：非常简单，直接用 percolate up / down 实现即可。

2. Delete: 把指定记录的键降为严格小于所有正常键的值(也可以标记为一个特殊的 $-\infty$), percolate up 到根结点后调用 DeleteMin。

这里还有一个前提：我们已经知道要修改或删除的元素在哪里。堆只保证父结点的键不大于孩子的键，不能像二叉搜索树那样根据键值选择左、右子树。因此，如果只给出元素的键值，却不知道它的位置，最坏情况下就需要检查整个堆，花费 $O(n)$ 时间。如果用数组保存堆，可以给每个元素分配一个不变的编号，再用一个辅助数组记录它当前的位置。例如， $pos[7] = 12$ 表示编号为 7 的元素在堆数组的第 12 个位置。这样，给出元素编号后，就能直接找到它。上滤或下滤时，每交换两个元素，都要把这两个元素在 pos 中的位置一起改好。如果用指针连接堆中的结点，也可以为每个元素保存一个指向其所在结点的指针。事实上，只要能用 $O(1)$ 时间找到目标元素，并在每次交换时用 $O(1)$ 时间更新它的位置，上述修改和删除操作就都能在 $O(\log n)$ 时间内完成。

看上去上面的操作需要的时间都十分完美，那么还需要在什么方面有改进呢？答案是有时候会考虑两个堆合并 (merge) 的问题。如果要对二叉堆进行合并，由于仍然需要保证合并后是完全二叉树，所以在基于数组的实现中，一个直接办法是拼接两个数组，然后调用 BuildHeap 在 $O(n_1 + n_2)$ 时间内完成；把较小堆逐项插入较大堆也是一种办法，其成本为 $O(\min(n_1, n_2) \log(n_1 + n_2 + 1))$ ，应视两个堆的大小选择。然而我们希望合并也是一个 $O(\log n)$ 内可以实现的操作，所以需要定义新的结构（也就是接下来要介绍的三种可合并堆）来实现这一点。

4.1.2 左式堆的定义与性质

下面首先介绍左式堆 (leftist heap)，这一数据结构仍然保持堆的序性质，但不再要求完全二叉树的结构性质。和它的名字一样，左式堆应当是整体向左倾斜的，那么如何严格定义这样的直观呢？这样的直观又有什么好处呢？下面来逐一介绍。首先需要定义一个 null path length 的概念以便于之后进一步定义左式堆的结构：

定义 4.1. 把任一结点 X 的零路径长 (null path length, NPL) $Npl(X)$ 定义为从 X 到一个没有两个儿子的结点的最短路径的长。因此，具有 0 个或 1 个儿子的结点的 Npl 为 0，且规定 $Npl(\text{null}) = -1$ 。

根据这一定义可知，对于一个堆（或者二叉树），计算每个结点的 Npl 应当从叶子结点出发向上计算，因为每个结点的 Npl 就等于它的两个孩子的 Npl 的最小值 + 1。这一结论适用于没有两个孩子的结点，因为定义中 $Npl(\text{null}) = -1$ 。

定义 4.2. 左式堆的结构性质是：每个结点的左孩子的 Npl 都要大于等于其右孩子的 Npl 。注意这一定义适用于没有两个孩子的结点，因为有定义 $Npl(\text{null}) = -1$ 。

图 4.1 给出两个例子，结点内写的是 NPL。右图虽然可以满足堆序（键值未写出，但可以构造符合要求的键值），但根的左孩子处，其左、右孩子 Npl 分别为 0 和 1，不满足左式堆的结构性质。

总结而言，我们基于 Npl 定义出了左式堆的结构性质，将向左倾斜的直观有了严格的表达。那么下一步需要研究的问题就是，向左倾斜有什么好处？下面的定理给出了形式化的结论，其中右路径指从根结点出发一路找右孩子直到右孩子为空的路径。



图 4.1 结点内标 NPL: 左图满足左式性质, 右图左孩子处不满足。

定理 4.3. 在右路径上有 r 个结点的左式堆必然至少有 $2^r - 1$ 个结点。

证明. 首先将 NPL 与右路径联系起来。由左式性质, $\text{Npl}(x) = \text{Npl}(x.\text{right}) + 1$ 。一直沿右孩子走到空结点, 可得非空堆根的 NPL 等于右路径结点数减一。

下面按树的结点总数归纳。考虑 $r = 0$, 其结点数显然至少为 $0 = 2^0 - 1$ 。若 $r > 0$, 根结点的右子树的右路径有 $r - 1$ 个结点; 左子树的 NPL 不小于右子树, 故左子树的右路径也至少有 $r - 1$ 个结点。对两棵规模更小的子树应用归纳假设, 各至少有 $2^{r-1} - 1$ 个结点, 加上根得到 $2^r - 1$ 。□

从这个定理立刻得到, 共有 n 个结点的左式堆的右路径最多含有 $\lfloor \log(n + 1) \rfloor$ 个结点。因此我们的想法是, 将左式堆的操作的所有的工作都放到右路径上进行, 因为它是 $O(\log n)$ 的。当然很显然的一点是, 对右路径的 Insert 和 Merge 可能会破坏左式堆性质, 但接下来的分析将表明, 恢复左式堆的性质是非常容易的。

有一个直觉是值得特别说明的, 就是平衡搜索树中通常要求树越平衡越好, 但堆却似乎不需要这一点, 这是为什么呢。这是因为堆不支持 find 操作, 所以左式堆左边的结点在操作中可以完全不被访问, 而接下来的讨论会表明只在右路径上操作也完全可以解决插入、删除最小值和合并, 因此完全不需要保持树的平衡。

4.1.3 左式堆的操作与实现

首先解决三个核心操作: Insert、DeleteMin 和 Merge。首先 Merge 是最核心的操作, 也是另外两个操作的基础: Insert 可以视为一个堆和一个单结点的堆的 Merge, 因此问题转化为 Merge; DeleteMin 首先删除根结点, 然后将根结点的两个子树 Merge, 因此关键问题还是 Merge。所以这三种操作最终都归结于 Merge, 时间复杂度也完全来源于 Merge, 所以接下来只对 Merge 进行详细的分析。下面介绍两个版本的解决方案, 其一是递归, 其二是迭代, 事实上它们从结果上来看是等价的, 我们分别来看它们在分析中的优劣。

递归实现

递归实现分为如下步骤:

1. 如果两个堆中至少有一个是空的, 那么直接返回另一个即可;
2. 如果两个堆都非空, 比较两个堆的根结点 key 的大小, key 小的是 H_1 , key 大的是 H_2 ;
3. 如果 H_1 只有一个顶点 (根据左式堆定义, 只要没有左孩子就一定是单点), 直接把 H_2 放在 H_1 的左子树就完成任务了; 很容易验证这样得到的结构符合左式堆性质, 此时 Npl 也没有变化;

4. 如果 H_1 不只有一个顶点, 则将 H_1 的右子树和 H_2 合并 (这是递归的体现, 在 base case 设计良好, 其它步骤也都合理的情况下你完全可以相信这一步递归帮你做对了), 成为 H_1 的新右子树;
5. 如果 H_1 的 Npl 性质被违反, 则交换它的两个子树;
6. 更新 H_1 的 Npl, 结束任务。

第 4 步先在右子树中递归, 第 5 步只在当前根处交换它的左右孩子。递归返回时, 两棵子树各自已经是左式堆; 交换不会改变它们内部的结构, 也不会改变根比孩子小的堆序。

算法 4.1 左式堆合并 LeftistMerge(a, b)

输入: 两棵左式堆根;

输出: 返回合并后的根

```

1: if  $a = null$  then
2:   若  $b \neq null$ , 令  $b.parent \leftarrow null$ ; return  $b$ 
3: end if
4: if  $b = null$  then
5:    $a.parent \leftarrow null$ ; return  $a$ 
6: end if
7: if  $a.key > b.key$  then
8:   交换  $a, b$ 
9: end if
10:  $a.right \leftarrow \text{LeftistMerge}(a.right, b)$ 
11: if  $npl(a.left) < npl(a.right)$  then
12:   交换  $a.left, a.right$ 
13: end if
14:  $a.npl \leftarrow npl(a.right) + 1$ 
15: 对  $a$  的每个非空孩子  $c$ , 令  $c.parent \leftarrow a$ 
16:  $a.parent \leftarrow null$ ; return  $a$ 

```

下面将图 4.2 的两个堆的合并过程逐步展开, 图 4.3 给出了第一步及其交换子树的步骤。



图 4.2 合并前的两个左式堆。两条右路径分别为 3, 8 和 6, 7, 18。

1. 首先比较发现 $3 < 6$, 因此递归要求 H_1 的右子树 H'_1 与 H_2 合并, H_1 的左子树不动;
2. H'_1 的根结点 $8 > 6$, 因此递归要求 H_2 的右子树 H'_2 与 H'_1 合并, H_2 的左子树不动;
3. H'_1 根结点 8 大于 H'_2 根结点 7, 因此递归要求 H'_2 的右子树 H''_2 与 H'_1 合并, H'_2 的左子树不动;

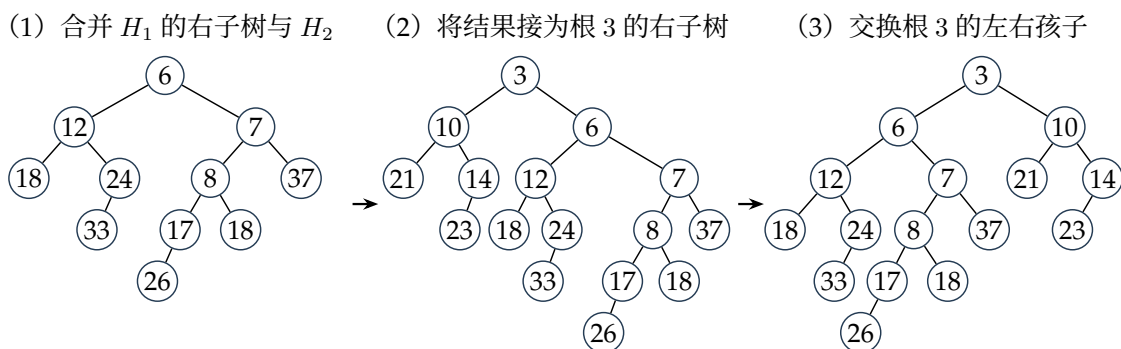


图 4.3 递归合并的三个步骤。第 (1) 步返回的子树已经满足左式性质；接回根 3 后，左右孩子的 NPL 分别为 1, 2，因此还需交换根 3 的左右孩子。

4. H_1' 根结点 8 小于 H_2'' 根结点 18，因此递归要求 H_1' 的右子树 H_1'' 与 H_2'' 合并， H_1' 的左子树不动；
5. 发现 H_1'' 是 null，因此直接连接上 H_2'' 即可，递归开始返回；
6. 递归返回，将 merge 后的树接到父亲的右子树上，然后判断是否需要交换子树并且更新 Npl 即可继续返回上一层递归，直到所有递归函数都返回就实现了整个过程。

根据上面的过程，不难分析算法 4.1 的时间复杂度：事实上，递归过程实际上就是沿着两个堆的右路径不断向下，递归返回的每一层也只需要常数时间即可完成，根据左式堆右路径长度性质，可知算法的时间复杂度是对数级别的。

迭代实现

我们可以进一步将抽象的递归算法展开成迭代形式。每一次递归向下时，我们应当保留根结点更小的堆的左子树，也就是左子树不动，右子树进入下一轮迭代，如此循环，直到最后一次与 null 合并；递归返回过程则是逐个检查新的右路径上的结点是否有违反 Npl 性质的并且更新 Npl 即可。根据这一过程，我们不难发现迭代版本的实现可以分为如下两步；(1) 将两个堆的右路径按键值从小到大排序，形成新堆的右路径，同时保留每个结点的左子树；(2) 从右路径的末端开始逐个检查是否有违反 Npl 性质的，如果有则交换左右孩子，最后更新 Npl 即可。图 4.4 给出了迭代版本的合并过程，具体的代码实现比较基本，读者可以自行尝试完成。

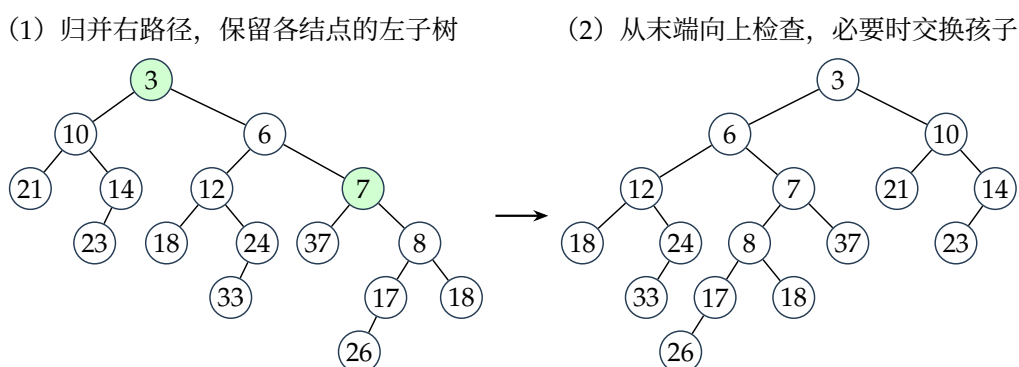


图 4.4 图 4.2 中两个堆的迭代合并。先将右路径 3, 8 和 6, 7, 18 归并为 3, 6, 7, 8, 18；再从末端向上更新 NPL，在浅绿色结点 7 和 3 处交换左右孩子，得到右图。

4.1.4 给定结点的删除与降键

设 u 是堆中的一个结点，删除和降键都可以归结为前面的合并操作：

- **删除 u** ：首先拆下以 u 为根的子树，修复剩余堆；然后去掉 u ，再将它的左右子树合并回剩余堆。
- **降低 u 的键**：如果新键仍不小于父亲的键，或者 u 就是根，直接修改即可；否则拆下以 u 为根的子树，修复剩余堆，再将降键后的子树合并回去。降低 u 的键不会破坏它与孩子的堆序，因此拆下的子树仍是合法左式堆。

自然地，这两个操作共同需要解决的关键问题是：拆下以 u 为根的子树之后，如何修复剩余堆。注意 u 的父亲少了一棵子树，可能违反左式性质，其 NPL 也可能下降，进而影响更高的祖先。因此，我们从 u 的父亲开始向上检查，每到一个结点 p ，依次执行：(1) 若左孩子的 NPL 小于右孩子，交换两个孩子，恢复左式性质；(2) 更新 p 的 NPL；(3) 如果 p 的 NPL 没有变化，算法结束；否则继续检查父亲，直到根。注意第 3 步可以结束算法，是因为以 p 为根的子树已经修复，而它的 NPL 与原来相同，则它的父亲所依据的两个孩子的 NPL 因而都没变，左式性质也就不变，更高祖先同理。

接下来讨论修复的时间复杂度，其核心是证明向上修复不会访问过多结点。设当前结点为 p ，刚被拆空或修复的子树为 C ，则 p 的 NPL 只可能下降。因此，若 C 是右子树，则无需交换， C 保留在右边；若 C 是左子树，只要它的 NPL 仍不小于右孩子， p 的 NPL 就不变，修复停止；若要继续向上，必然需要交换孩子，把 C 放到右边。所以，每当修复继续向上进行，刚修复的下层子树都会成为当前结点的右子树。因此，修复的次数至多是修复后的右路径长度，因此时间复杂度是对数级别的。

4.2 斜堆

4.2.1 简介

斜堆与左式堆的关系类似于 Splay 树和 AVL 树之间的关系。回顾 Splay 树，它并不需要维护 AVL 树中的 bf 属性，只需要在访问一个结点之后就无脑地将它用 zig / zig-zig / zig-zag 三种情况将它翻到根结点即可。斜堆的想法类似，它不用再维护 Npl，因此在递归过程中左式堆所有维护结构性质以及更新 Npl 的操作不再需要，取而代之的是如下操作：

1. 在 base case 处理 H 与 null 连接的情况时，左式堆直接返回 H 即可，但斜堆必须看 H 的右路径，要求 H 右路径上除了最大结点之外都必须交换其左右孩子。
2. 在非 base case 时，若 H_1 的根结点小于 H_2 ，如果是左式堆，需要合并 H_1 的右子树和 H_2 作为 H_1 的新右子树，最后再判断这样是否违反性质决定是否交换左右孩子，斜堆直接无脑交换，也就是说每次这种情况都把 H_1 的左孩子换到右孩子的位置，然后把新合并的插入在 H_1 的左子树上。

根据上述描述以及与左式堆的对比，斜堆的合并本质上就是将右路径排序后，沿右路径逐个交换左右孩子，但注意最后一个结点的左右孩子不交换。所以类似于左式堆，最后合并出的堆的左路径上包含两个原始堆的右路径排序后的结果，当然后面还可能连着原始堆右路径最大值的一些左孩子（因为这些左孩子是不被交换的）。图 4.5 基本使用图 4.2 的两个输入堆进行展示，区别在于 18 添加了左孩子 35。此时的合并路径是 3, 6, 7, 8, 18，前四个结点交换孩子，最后一个 18 不交换，35 仍是它的左孩子。需要注意的是，有的地方将斜堆的 base case 定义为与左式堆一致，但在这门课中都按照上面描述的方式实现。此外，插入和删除最小值仍然是通过合并实现的（图 4.6 展示了在图 4.5 的结果上

进一步插入 15 的结果); 至于任意结点 u 的删除, 可以首先合并 u 的左右子树, 用合并结果替换 u ; 对于任意结点的降键, 一种直接实现是先删除该结点, 再按新键重新插入。

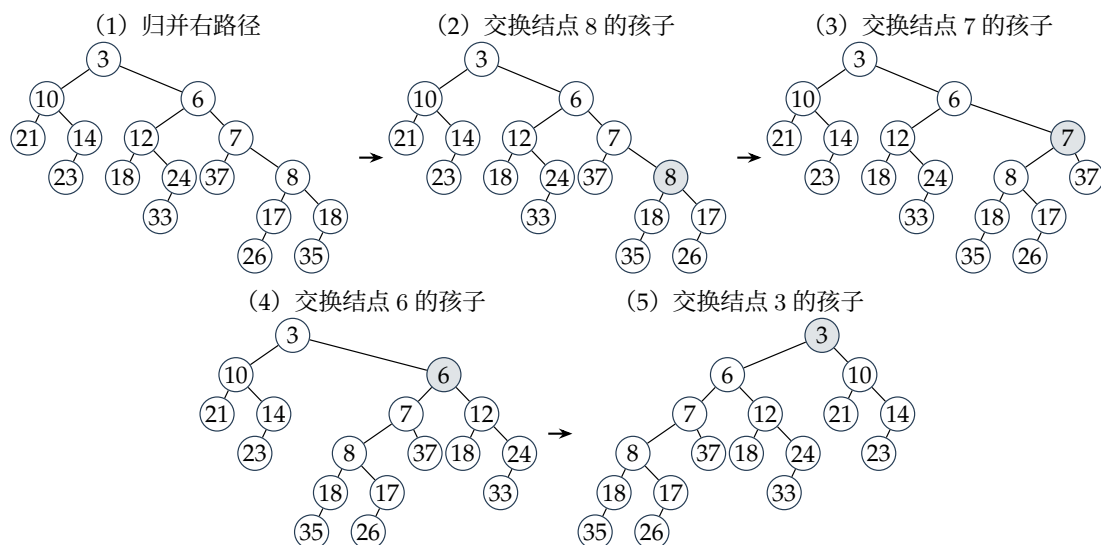


图 4.5 斜堆合并：先归并右路径，再从末端向上交换孩子。合并路径为 3, 6, 7, 8, 18，末端 18 不交换，保留左孩子 35；依次在 8, 7, 6, 3 处交换，得到第 (5) 步的结果。浅色结点表示当前交换的位置。



图 4.6 插入 15: 合并路径为 3, 10, 14, 15, 依次交换 14, 10, 3 的孩子, 末端 15 不交换 (实际上 15 也并没有子树)。方框 U 表示图 4.5 最终结果中以 6 为根的整棵子树。

4.2.2 摊还分析

根据上面的介绍，我们发现斜堆的好处很多，例如不需要多余空间存储 N ，也不需要有很多判断、更新操作。然而，斜堆的定义没有限制住右路径的长度，因此无法像左式堆那样直接用右路径长度 bound 住它的时间复杂度。所以一个自然的问题是，它真的能像我们希望的那样，像 Splay 树一样具有 $O(\log n)$ 的摊还操作代价吗？接下来开始分析。首先需要明确的一点是，Insert 和 DeleteMin 还是以 Merge 为核心，所以一系列 Insert、DeleteMin、Merge 操作分析的关键还是 merge，因此只需分析 merge。

下面仍用势函数法分析。为了定义合适的势函数，需要给出一个辅助的定义：

定义 4.4. 称一个结点 P 是**重的 (heavy)**, 如果它的右子树结点个数至少是 P 的所有后代的一半 (后代包括 P 自身)。反之称为**轻结点 (light node)**。

图 4.7 标出一次合并前后的轻重状态。判断一个结点的轻重时，需要比较它的右子树大小与整棵子树大小。

为了证明后面的主要定理, 在此先给出一个引理:

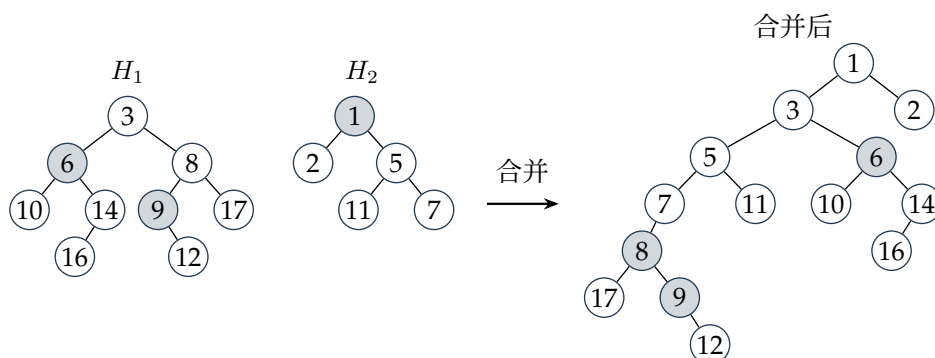


图 4.7 斜堆合并前后的轻重状态：灰底表示重结点，白底表示轻结点。合并路径上的 1 由重变轻，8 由轻变重；路径外的 6, 9 保持为重结点。

引理 4.5. 对于右路径上有 l 个轻结点的斜堆，整个斜堆至少有 $2^l - 1$ 个结点，这意味着一个 n 个结点的斜堆右路径上的轻结点个数为 $O(\log n)$ 。

证明. 这里的证明和前面证明左式堆性质是十分类似的。对于 $l = 1$ 显然成立，现在设小于等于 l 都成立，对于 $l+1$ 的情况，找到右路径上第二个轻结点，那么它所在子树大小根据归纳假设至少有 $2^l - 1$ 个结点。现在考虑第一个轻结点，根据轻结点定义它的左子树更大，而右路径上第二个轻结点所在的子树在其右子树中，因此它的左子树至少有 $2^l - 1$ 个结点。故整个堆至少有 $1 + (2^l - 1) + (2^l - 1) = 2^{l+1} - 1$ 个结点。□

定理 4.6. 若有两个斜堆 H_1 和 H_2 ，它们分别有 n_1 和 n_2 个结点，则合并 H_1 和 H_2 的摊还时间复杂度为 $O(\log n)$ ，其中 $n = n_1 + n_2$ 。

证明. 定义势函数 $\Phi(H_i)$ 等于堆 H_i 的重结点的个数，并令 H_3 为合并后的新堆。设 $H_i (i = 1, 2)$ 的右路径上的轻结点数量为 l_i ，重结点数量为 h_i ，因此真实的合并操作最坏的时间复杂度为 $c_i = l_1 + l_2 + h_1 + h_2$ （所有操作都在右路径上完成）。因此根据摊还分析可知摊还时间复杂度为

$$\hat{c}_i = c_i + \Phi(H_3) - (\Phi(H_1) + \Phi(H_2)).$$

事实上，在合并前可以记

$$\Phi(H_1) + \Phi(H_2) = h_1 + h_2 + h,$$

其中 h 表示不在右路径上的重结点个数。现在要考察合并后的情况，有两个非常重要的观察：

1. 只有在 H_1 和 H_2 右路径上的结点才可能改变轻重状态，这是很显然的，因为其余结点的子树在合并前后保持不变，所以轻重状态也不变；
2. H_1 和 H_2 右路径上的重结点在合并后一定会变成轻结点，这是因为右路径除最后一个结点外一定会交换左右子树，而最后一个结点原来没有右孩子，本来就是轻结点。

结合以上两点可知，合并后原本不在右路径上的 h 个重结点仍然是重结点，在右路径上的 $h_1 + h_2$ 个重结点全部变成轻结点， $l_1 + l_2$ 个轻结点不一定都变重，因此合并后有

$$\Phi(H_3) \leq l_1 + l_2 + h,$$

代入数据计算可得

$$\hat{c}_i \leq (l_1 + l_2 + h_1 + h_2) + (l_1 + l_2 + h) - (h_1 + h_2 + h) = 2(l_1 + l_2).$$

根据前面的引理, $l_1 + l_2 = O(\log n_1 + \log n_2) = O(\log(n_1 + n_2)) = O(\log n)$, 其中第二个等号利用了 $\log n_1 \leq \log(n_1 + n_2)$ 和 $\log n_2 \leq \log(n_1 + n_2)$ 。且注意到初始(空堆)势函数一定为 0, 且之后势函数总是非负的, 所以这一势函数定义满足要求, 证明也就完成了。□

这个势函数体现了前面强调的构造思路: 让昂贵操作带来较大的势能下降。当合并成本很高时, 两条右路径上必然有很多结点, 其中轻结点的数量已有对数上界, 所以多出来的主要是重结点。合并后, 这些重结点全部变轻, 重结点数的下降便抵消了遍历长路径的成本。

需要说明的是, 有一种很直观的势函数定义方法在此是行不通的。我们知道, 一次合并的效果是右路径上的每一个结点都被移到左路径上, 而其原左儿子变成新的右儿子。所以自然的一种想法是把每一个结点算为右结点或左结点来分类, **这是根据结点是右儿子, 或不是右儿子来决定的**。将右结点的个数作为势函数, 虽然这一势函数初始为 0 并且总是非负, 但是问题在于, 如果考虑一种最坏的情况, 就是右路径长度为 $\Theta(n)$ 时, 我们希望势函数减小很多使得摊还代价是对数的, 然而很显然能找到例子使得在这种情况下操作之后右结点数量不变, 所以这种情况下右结点并不适合于解决问题。因此考虑另一种基于轻重的分类来解决斜堆的摊还分析问题。

4.3 习题

习题 4.1 (NPL 与右路径长度). 非空左式堆根结点的 NPL 是否等于右路径长度(长度按边数计算)?

习题 4.2 (NPL 会增加多少). 两棵左式堆的根 NPL 不相同, 合并后的根 NPL 是否一定不超过两者较大值? 是否一定不超过较小值加一? 用同一个反例回答。

习题 4.3 (摊还界不是精确成本). 构造由两棵非空斜堆组成的输入, 使一次合并需要线性时间, 并且“右孩子总数”这一候选势函数在合并过程中不下降。

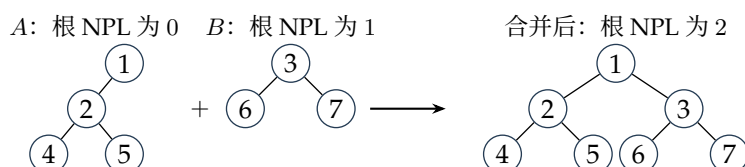
习题 4.4 (从小到大插入). 设 $k \geq 1$, 从空堆开始, 按键值从小到大依次插入 $1, 2, \dots, 2^k - 1$ 。判断以下说法是否正确, 并给出证明或反例(无需证明两棵树的键值布局相同):

1. 对左式堆执行上述操作, 结果是一棵有 k 层、每层都满的完美二叉树。
2. 对斜堆执行上述操作, 结果也是一棵有 k 层的完美二叉树。

4.4 参考答案

习题 4.1. 相等。因为由左式性质, 任意结点 x 都有 $\text{Npl}(x) = \text{Npl}(x.\text{right}) + 1$ 。□

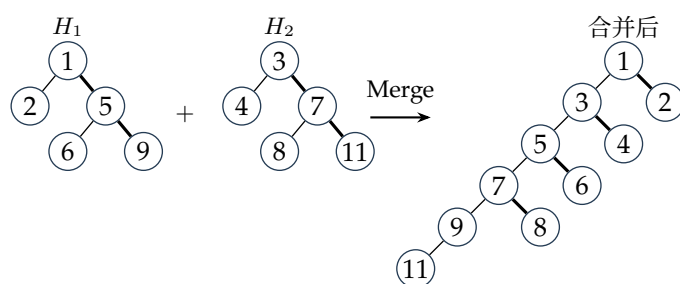
习题 4.2. 如图所示, 两堆根的 NPL 分别为 0, 1, 合并后根的 NPL 为 2, 同时否定两个说法。



□

习题 4.3. 对任意 $q \geq 2$, 构造两棵非空斜堆 H_1, H_2 。 H_1 的右路径为 $1, 5, 9, \dots, 4q - 3$, 并且对 $i = 0, 1, \dots, q - 2$, 给右路径上的结点 $4i + 1$ 添加一个左叶子 $4i + 2$ 。类似地, H_2 的右路径为 $3, 7, 11, \dots, 4q - 1$, 并且对 $i = 0, 1, \dots, q - 2$, 给结点 $4i + 3$ 添加一个左叶子 $4i + 4$ 。显然两棵树都满足最小堆序, 因而都是合法斜堆。两条右路径各有 q 个结点。合并时, 它们恰好交错归并为 $1, 3, 5, 7, \dots, 4q - 1$, 所以合并路径共有 $2q$ 个结点。两棵输入堆总共有 $(2q - 1) + (2q - 1) = 4q - 2$ 个结点, 因此这次合并的实际成本为 $\Theta(q) = \Theta(n)$ 。

再看右孩子总数。合并前, 两棵堆的右路径各贡献 $q - 1$ 个右孩子, 所以总数为 $2q - 2$ 。合并后, 除合并路径的最后一个结点外都交换左右孩子; 原来挂在合并路径上的 $2q - 2$ 个左叶子恰好全部变成右孩子。因此合并后仍恰有 $2q - 2$ 个右孩子。所以这次线性时间的合并前后, 相应的势函数完全没有下降, 它无法抵消 $\Theta(n)$ 的实际成本。下图给出 $q = 3$ 的例子。



□

习题 4.4. 两个说法都正确, 证明只需要找规律使用归纳法即可。

□

参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第四讲。
2. Mark Allen Weiss, 《数据结构与算法分析: C 语言描述》, 第二版, 第 6 章的左式堆、斜堆, 以及第 11 章的斜堆摊还分析。